Birla Institute of Technology and Science, Pilani

Hyderabad Campus

RepoRAG Project Report

CS F469 – Information Retrieval

Anshul Rath (2023A7PS0084H)

Devansh Agarwal (2023A7PS0044H)

July 11, 2026

1 Introduction

Modern software development involves large, complex, and highly interdependent codebases. As a result, developers, especially new contributors, often face a steep onboarding challenge. A significant amount of time is spent understanding existing code rather than building new features or fixing bugs. This highlights the need for tools that provide intuitive and contextual understanding of codebases.

Traditional navigation techniques such as grep or keyword-based search fall short in this setting. They rely on exact text matching and fail to capture semantic meaning or structural relationships within the code. Consequently, developers are often presented with fragmented results and must manually reconstruct the underlying logic.

The core problem is that keyword-based systems cannot effectively handle the structural dependencies present in modern software systems. Applications span multiple files, classes, and modules, making it difficult to answer high-level queries such as how different components interact or how specific functionalities are implemented.

To address this, we introduce **RepoRAG**, a context-aware system designed for intelligent codebase exploration. By leveraging semantic understanding and dependency analysis, RepoRAG enables users to ask natural language questions and receive coherent, contextually relevant explanations. This approach reduces onboarding time and enhances developer productivity by making codebases more accessible and understandable.

2 Methodology

2.1 Overview of the Complete Pipeline

The proposed system follows a structured pipeline that transforms a raw code repository into an intelligent, queryable system. The pipeline is divided into **three key stages: indexing and structuring, query processing with hybrid retrieval, and context-aware response generation**. Subsequent sections detail each stage and its components.

2.1.1 Indexing and Structuring

The first stage focuses on converting the codebase into a structured and machine-understandable format. The repository is parsed using Abstract Syntax Trees (ASTs) to capture syntactic and functional elements such as functions, classes, and variables.

From these parsed representations, vector embeddings are generated to encode the semantic meaning of code components. In parallel, a dependency graph is constructed to model relationships between different parts of the codebase, enabling the system to understand how various functions interact.

2.1.2 Query Processing and Hybrid Retrieval

User queries are first processed to improve clarity and retrieval effectiveness. Ambiguous or high-level queries are expanded using a lightweight language model, incorporating relevant software engineering terminology.

The system then performs hybrid retrieval by combining dense vector-based search with

sparse keyword-based retrieval (BM25). These two retrieval signals are fused using Reciprocal Rank Fusion (RRF), which aggregates rankings rather than raw scores.

2.1.3 Context Assembly and LLM Synthesis

After retrieval, the system extracts the immediate neighborhood of the retrieved code snippets from the dependency graph. A lightweight language model, using few-shot prompting, is employed to select whether to include neighbouring nodes from the dependency graph. This is explained in more detail in the next section.

The selected code snippets and their structural context are then provided to a language model, which generates a coherent and context-aware response. This enables the system to deliver precise explanations, summaries, or code insights tailored to the user's query.

2.2 Indexer: Function Extraction and Representation

The indexing module transforms a code repository into a structured representation using AST parsing and semantic embeddings.

2.2.1 AST-based Function Extraction

The repository is parsed to build Abstract Syntax Trees (ASTs). These help capture syntactic and functional elements such as functions, classes, methods and relationships like function calls and imports. This structured representation allows us to build retrieval system that can fetch relevant code snippets.

We make use of the `tree-sitter` library, which contains language-specific parsers for various programming languages. Using this, we have added support for languages such as **Python, JavaScript, Java, C, C++ and TypeScript**. This library, using the language specific modules, helps builds ASTs for the source code files. We then use language specific queries to extract function and method definitions, along with their source code, line numbers and docstrings. We also extract import statements (languages such as Python, JavaScript, Java, etc involve imports which help us in mapping function calls to exact definitions) and call relationships to build a dependency graph.

Each function is stored as a `FunctionNode` containing a unique identifier (built using the metadata of the function), source code, line numbers, and optional docstring. The function names are stored as fully qualified names, which includes the class names to represent method definitions. This helps in clearly identifying functions in the codebase.

2.2.2 Import and Call Extraction

An import map is built for each file, linking symbols to modules. These import statements are extracted using relevant tree-sitter queries. Function bodies are analyzed to extract call relationships, such as function calls, method invocations, and decorators functions. This is also done using language-specific queries to identify call expressions and their targets. These calls are stored in the same function node as a list of called function names, which are later used to build the dependency graph.

2.2.3 Semantic Embedding Generation

Each function is converted into an embedding using its name, source code, docstring and other metadata. We use a pre-trained code embedding model, `jina-embeddings-v2-base-code`, which is a multilingual embedding model supporting English and 30 programming languages. It has a sequence length of 8192 tokens. Functions that exceed this length are split into overlapping chunks, embedded separately, and averaged into a normalized vector, enabling dense retrieval.

The complete indexer pipeline thus outputs a list of function nodes, a global import map, and embeddings for each function.

2.3 Dependency Graph Construction

A directed graph is built where nodes represent functions and edges represent call relationships. This graph has been constructed using the `networkx` library.

2.3.1 Graph Construction

Each function is added as a node with metadata, which includes the function identifier, fully qualified name, source code, docstrings and line numbers. The edges are created based on the call relationships extracted during indexing.

2.3.2 Edge construction

The import map is used to resolve method calls across files by matching class or module names with imported symbols or local definitions. The edges are created using fuzzy matching to handle cases where it is difficult to disambiguate the exact function defination being called. The edges are associated with certain weights that represent the confidence of the match.

Edge Weighting:

Edges are weighted based on confidence. The cases are as follows:

- **Exact Match:** If the called fully qualified function name matches exactly with the function in the repository in the same language, the edge is assigned a high weight (1.0), otherwise, if the function name matches but the language is different, the edge is assigned a moderate weight (0.5). If the function name matches but with the base name only (without class or module), the edge is assigned a weight of 0.6 if the language matches, and 0.3 if the language does not match.
- **Import Match:** If the function is a method call, the base name is matched in a dictionary which maps the base names to the fully qualified names. The class/module names are extracted from the identifiers of the method call extracted from the mapping and matched with the imports of the file the method call is present in. If there is a match, the edge is assigned a weight of 0.9 if the exact import is found and 0.2 if the language does not match.

2.3.3 Final Graph

The resulting graph captures structural dependencies and supports context extraction for downstream processing. The graph is stored on the disk in a serialized format in JSON,

which can be loaded later during retrieval.

2.4 Retriever

The retrieval module identifies relevant code snippets for a given query using both sparse (BM25) and dense (embedding-based) techniques.

2.4.1 Query Expansion

The user query is expanded with related software engineering terms to improve retrieval coverage.

A lightweight language model is used to generate a small set of relevant technical synonyms, library names, or programming concepts associated with the query. The model is constrained to return a structured list of terms, which are then appended to the original query.

This expanded query improves recall by bridging vocabulary gaps between natural language queries and code. For example, a query about database access may be augmented with terms such as specific libraries or related concepts, enabling better matching during both BM25 and dense retrieval.

2.4.2 BM25-based Sparse Retrieval

BM25 is used as the primary lexical retrieval method to match query terms with code tokens.

Each function's source code is tokenized using a code-aware tokenizer that:

- Splits camelCase and snake_case identifiers
- Removes punctuation
- Converts text to lowercase

This preprocessing improves alignment between natural language queries and code elements such as function names and variables.

Given a query, BM25 computes relevance scores based on term frequency and inverse document frequency, returning the top-ranked functions. Since natural language queries may not always match code tokens directly, BM25 serves as a strong baseline for lexical retrieval. Most of the retrieved results are expected to be matches based on the function names and associated docstrings which contain natural language descriptions of the function's purpose.

2.4.3 Dense Retrieval

Dense retrieval captures semantic similarity between queries and code. This is achieved using the embedding model discussed in the previous section. This model is trained on a large code corpus along with natural language descriptions, enabling it to generate embeddings that reflect the meaning of code snippets.

The user query is converted into a vector embedding using the same model used during indexing. This embedding is compared with all function embeddings using cosine similarity.

Functions are ranked based on similarity scores, enabling retrieval even when there is minimal lexical overlap between the query and the code.

2.4.4 Hybrid Retrieval

To combine sparse and dense retrieval, we use Reciprocal Rank Fusion (RRF) to merge BM25 and embedding-based rankings.

Given a query, two rankings are generated:

- BM25 ranking (which captures lexical similarity)
- Dense ranking (which captures semantic similarity)

These are combined using:

$$\text{RRF}(d) = \sum \frac{1}{k + \text{rank}(d)},$$

where k is a constant (taken to be 60 in our implementation) and $\text{rank}(d)$ is the rank of document d in each individual ranking. The final score for each document is the sum of its reciprocal ranks across both rankings.

This rank-based fusion avoids dependence on score scales and produces a balanced final ranking.

2.4.5 Neighborhood Expansion

After top-ranked functions are retrieved using the hybrid retrieval, we expand the context by including neighboring nodes from the dependency graph. The dependency graph constructed during the indexing phase is traversed using `networkx` to identify immediate neighbors (functions that call or are called by the retrieved functions).

`networkx`'s built-in function, `pagerank`, is used to rank the neighboring nodes based on their connectivity and edge weights in the graph. This essentially performs a multi-source BFS traversal from the retrived nodes, where the traversal is guided by the edge weights. The top-ranked neighboring nodes are selected to be included in the context for response generation.

In subsequent sections of evaluation, we will discuss the impact of neighbourhood expansion on the quality of generated responses with both lexical and semantic retrieval.

2.4.6 Query Classifier

A lightweight language model is used to classify whether the user query requires neighborhood expansion. This is done using few-shot prompting, where the model is given examples of queries that do and do not require neighborhood expansion. The model then predicts whether the current query would benefit from including neighboring nodes in the context. This allows the system to dynamically decide when to include additional context, improving response relevance without overwhelming the generator language model with unnecessary information.

2.5 Generator

The generator produces the final response using the retrieved code context.

2.5.1 Context Construction

Retrieved functions are formatted into a structured context containing function name, file path, and source code. The codebase context is appropriately formatted in cases where dependency graph traversal is performed, ensuring that the relationships between functions are clearly represented. This structured context serves as the input for the language model to generate a response.

2.5.2 Response Generation

The final input consists of the user query and the constructed code context. A system prompt is generated to guide the language model in generating a response. It contains all the relevant instructions for the language model to follow while answering subsequent queries asked in the user prompt. The language model is asked to generate a response grounded strictly in this context, ensuring relevance to the retrieved code.

3 Evaluation

As any evaluation of a RAG system, the evaluation of the current system is split into two parts: retrieval evaluation and generation evaluation. The retrieval evaluation focuses on assessing the relevance of the retrieved code snippets to the user query, while the generation evaluation assesses the quality and accuracy of the generated responses based on the retrieved context.

A ground truth dataset is constructed for evaluation, consisting of a set of user queries and their corresponding relevant code snippets. This dataset is used to evaluate both the retrieval and generation components of the system.

The evaluation makes use of LLM-as-a-Judge to judge the relevance of generated responses to the user queries. The LLM is prompted with the user query, the retrieved code snippets, and the generated response, and is asked to evaluate the relevance of the response based on the retrieved context. Further details of the evaluation are discussed in the subsequent sections.

Furthermore, we also evaluate our system against an Agentic RAG system, which uses an agent to iteratively retrieve and generate responses based on the user query. This allows us to compare the performance of our RAG system with the Agentic RAG approach.

3.1 Ground Truth Construction

A ground truth dataset is constructed for evaluation, consisting of a set of user queries and their corresponding relevant code snippets. We made use of a code repository that we had worked on in the past. The indexer pipeline extracted 95 functions from the repository. The total execution time for indexing the complete repository took 25.35 seconds.

The ground truth dataset was constructed using a large language model, which was prompted with a particular codebase context and asked to generate queries whose answer would require retrieveing that particular code snippet. The queries constructed using the model are categorised into two types:

1. **Direct Queries:** These queries can be answered by retrieving the relevant code snippet directly, without requiring any additional context from the dependency graph. For example, a query asking about the functionality of a specific function can be answered by retrieving that function's code snippet and its docstring. These queries are labelled as **single-hop queries** since they can be answered by retrieving a single code snippet. These direct queries are further classified according to their type:
   (a) **Exact Match Queries:** These queries contain keywords and other short code snippets from the codebase context provided. The aim of these queries is to check if exact-match retrieval using BM25-only retrieval is able to retrieve relevant code snippets from the codebase.
   (b) **Semantic Match Queries:** These queries do not contain exact keywords or short code snippets from the codebase context provided. Rather, the LLM tries to understand the meaning of the codebase context and constructs a query that can be perfectly answered using semantic retrieval. The aim of these queries is to check if semantic retrieval using embedding-only retrieval is able to retrieve relevant code snippets from the codebase.
   (c) **Structural Queries:** These queries require understanding the structure of the codebase to answer effectively. They don't necessary involve retrieving multiple code snippets, but they require understanding the codebase structure to be able to retrieve the relevant code snippet.
2. **Contextual Queries:** These queries require additional context from the dependency graph to be answered effectively. For example, a query asking about how a particular function interacts with other functions in the codebase would require retrieving not only the relevant function's code snippet but also the code snippets of the neighboring nodes in the dependency graph. These queries are labelled as **multi-hop queries** since they require retrieving multiple code snippets from the codebase to be answered effectively.

Direct Queries are constructed by taking a particular functions code snippet and associated metadata. Then, using the prompts and structure explained before, the LLM generates the those three types of queries.

Contextual Queries require taking a particular function's code snippet and associated metadata, along with the code snippets and metadata of its neighboring nodes in the dependency graph. The neighbouring nodes in the graph is not limited to its children but also its parent nodes. Then, the LLM generates queries that require retrieving these multiple code snippets to be answered effectively.

The constructed ground truth dataset consists of 285 direct queries (95 exact match queries, 95 semantic match queries and 95 structural queries) and 85 contextual queries.

3.2 Retrieval Evaluation

The retrieval evaluation assess the performance of the retrieval system on the ground truth queries constructed. The evaluation metrics used are as follows:

- **Recall@k:** This metric measures the proportion of relevant code snippets that are retrieved in the top-k results. It is calculated as:

$$\text{Recall@k} = \frac{\text{Number of relevant code snippets retrieved in top-k}}{\text{Total number of relevant code snippets}}$$

For single-hop queries, the total number of relevant code snippets is 1, while for multi-hop queries, it is equal to the number of code snippets that are required to be retrieved to answer the query effectively. Thus, for single-hop queries, this metric can also be referred to as **Hit@k**.

- **Mean Reciprocal Rank (MRR):** This metric measures the average reciprocal rank of the first relevant code snippet retrieved. It is calculated as:

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^{N} \frac{1}{\text{rank of first relevant code snippet for query } i}$$

This is particularly relevant for single-hop queries, where we are interested in how high the relevant code snippet is ranked in the retrieval results.

- **nDCG@k:** This metric measures the normalized discounted cumulative gain at rank k, which takes into account the relevance of the retrieved code snippets and their positions in the ranked list. It is calculated as:

$$\text{nDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}}$$

where DCG@k is the discounted cumulative gain at rank k and IDCG@k is the ideal discounted cumulative gain at rank k.
This metric is particularly relevant for multi-hop queries, where we are interested in not only whether the relevant code snippets are retrieved but also how high are they ranked in the retrieval results.

These evaluation metrics are computed at different values of k such as 3, 5 and 7.

The results of the retrieval evaluation are displayed in the tables 1 and 2 for single-hop and multi-hop queries respectively. The results show the performance of different retrieval methods, including BM25-only retrieval, BM25 + Dependency Graph retrieval, and hybrid retrieval with and without neighborhood expansion.

Method	**MRR**	**R@3**	**R@5**	**R@7**
BM25	0.45	0.56	0.65	0.70
BM25 + Dep	0.44	0.55	0.66	0.70
BM25 + Dense	**0.67**	**0.76**	**0.84**	0.87
BM25 + Dense + Dep	0.44	0.60	0.80	**0.88**

Table 1: Comparison of retrieval methods using MRR metric for single-hop queries.

Method	**R@3**	**nDCG@3**	**R@5**	**nDCG@5**	**R@7**	**nDCG@7**
BM25	0.54	0.58	0.65	0.63	0.73	0.66
BM25 + Dep	0.68	0.68	0.78	0.74	0.82	0.76
BM25 + Dense	**0.76**	**0.77**	0.82	**0.80**	0.85	**0.82**
BM25 + Dense + Dep	0.71	0.68	**0.85**	0.75	**0.91**	0.78

Table 2: Comparison of retrieval methods using R@k and nDCG@k metrics for multi-hop queries.

The table 1 shows that the hybrid retrieval method (BM25 + Dense) significantly outperforms BM25-only retrieval in terms of MRR and Recall@k for single-hop queries. The

addition of the dependency graph does not seem to improve the performance for single-hop queries, which is expected since these queries do not require additional context from the graph. A slight increase in Recall@7 is observed when the dependency graph is added to the hybrid retrieval, which could be attributed to the fact that some relevant code snippets might be ranked higher due to the additional context provided by the graph.

The slight decrease in the evaluation metrics when dependency graph is added to retrieval methods can be attributed to the fact that the dependency graph might introduce some noise in the retrieval results, especially for single-hop queries where the relevant code snippet can be retrieved without requiring additional context from the graph. The dependency graph might also introduce some irrelevant code snippets that are not directly related to the query, which can negatively impact the evaluation metrics. The irrelevant code snippets that are retrieved can overwhelm the top-k results and push the relevant code snippets down the ranking, leading to a decrease in MRR and Recall@k. This is particularly evident in the case of BM25 + Dense + Dep method, where the addition of the dependency graph seems to have a negative impact on the evaluation metrics for single-hop queries.

This is why, we have introduced a **lightweight language model-based query classifier** to determine whether a particular query would benefit from including neighboring nodes from the dependency graph. This uses few-shot prompting to guide the classification process. This allows the system to dynamically decide when to include additional context.

The table 2 shows that the hybrid retrieval method (BM25 + Dense) also significantly outperforms BM25-only retrieval in terms of R@k and nDCG@k for multi-hop queries. The addition of the dependency graph further improves the performance for multi-hop queries, which is expected since these queries require more context from the graph. A particular improvement is observed in the Recall@5 and Recall@7 metrics when the dependency graph is added to the hybrid retrieval.

3.3 Generation Evaluation

The generator synthesizes responses based on the retrieved code context. The evaluation of the generator focuses on assessing the quality and accuracy of the generated responses in relation to the user queries and the retrieved code snippets. To separate the evaluation of generation from retrieval, we use the ground truth dataset of queries and their corresponding relevant code snippets as context for the generator to synthesize its response. This allows us to evaluate the generator's ability to produce accurate and relevant responses based on the provided context.

The models used for generation are:

- **llama-3.3-70b-versatile**: Accessed via the Groq API, this 70B parameter model from Meta serves as the high-capability baseline in our setup. Its large size enables strong instruction-following and code understanding, making it suitable for complex generation tasks.
- **gemma3:4b**: It offers efficient reasoning capabilities while remaining lightweight enough for local deployment.
- **phi4-mini:3.8b**: This model was chosen for its exceptional reasoning and coding ability despite its small footprint.

- **qwen2.5-coder:3b**: A code-specialized model from Qwen family, selected specifically for its focus on code generation and understanding.

The last three models were run locally using `ollama`.

The evaluation of the generated responses is performed using LLM-as-a-Judge. We made use of **gemma-4-31b-it** model accessed through Google AI Studio. This model was chosen because of its massive context window, which was particularly required for our purpose of evaluating the generated responses.

The generated responses are graded based on the following metrics:

- **Faithfulness**: Evaluates if the answer is completely grounded in the retrieved context with no hallucinations.
- **Completeness**: Checks if the answer fully addresses the query
- **Correctness**: Checks the technical accuracy of the answer, ensuring that it correctly interprets the code snippets and their relationships.
- **Clarity**: Scores based on how clear the answer is.

These metrics are scored on a scale of 1 to 5, where 1 indicates poor performance and 5 indicates excellent performance.

The results of the evaluation are displayed in the table 5 and 6. The results show the performance of different generation models on the ground truth queries. The evaluation is performed on **30 single-hop queries** and **20 multi-hop queries** from the ground truth dataset. The generated responses are evaluated based on the metrics mentioned above, and the average scores for each metric are reported in the tables.

Model	Faithfulness	Correctness	Completeness	Clarity	Total
Llama 3.3 70B Versatile (API)	5.000	4.933	4.867	5.000	19.800

Table 3: Scores of the Llama 3.3 70B Versatile model for single-hop queries.

Model	Faithfulness	Correctness	Completeness	Clarity	Total
Llama 3.3 70B Versatile (API)	4.90	4.85	4.85	4.80	19.00

Table 4: Scores of the Llama 3.3 70B Versatile model for multi-hop queries.

Model	Faithfulness	Correctness	Completeness	Clarity	Total
gemma3:4b (Local)	4.767	4.767	5.000	4.967	19.501
phi4-mini:3.8b (Local)	4.167	3.700	4.167	4.567	16.601
qwen2.5-coder:3b (Local)	4.933	4.833	4.967	4.967	19.699

Table 5: Comparison of generation models using LLM-as-a-Judge evaluation metrics for single-hop queries.

Note that tables 3 and 4 have been added as a baseline for comparing the local models. We note how good the local models perform when compared to this model.

From these tables we infer that among local models, **qwen2.5-coder:3b** performs the best in terms of generation quality for single-hop queries, closely following the performance of the larger **Llama 3.3 70B Versatile** model accessed via API. The performance of **gemma3:4b** closely follows that of **qwen2.5-coder:3b**.

The **qwen2.5-coder:3b** model performs well, particularly in terms of faithfulness and clarity, which suggests that it is effective at generating responses that are grounded in

Model	Faithfulness	Correctness	Completeness	Clarity	Total
gemma3:4b (Local)	4.00	3.90	4.10	4.30	16.30
phi4-mini:3.8b (Local)	3.40	3.50	4.35	4.35	15.60
qwen2.5-coder:3b (Local)	3.95	3.75	4.10	4.80	16.60

Table 6: Comparison of generation models using LLM-as-a-Judge evaluation metrics for multi-hop queries.

System	Relevance	Completeness	Precision
Agentic RAG	4.6	3.4	**3.26**
RepoRAG (Ours)	**4.8**	**4.67**	3.17

Table 7: Comparison of Context of Agentic RAG system and our system using LLM-as-a-Judge evaluation metrics for single-hop queries.

the retrieved code snippets and are easy to understand. These models, despite being significantly smaller than the Llama 3.3 70B Versatile model, demonstrate strong performance in generating accurate and relevant responses based on the retrieved code context, especially for single-hop queries.

3.4 Comparison with Agentic RAG

An Agentic RAG system is implemented using a simple agent that iteratively retrieves and generates responses based on the user query. The agent uses the same retrieval and generation components as our system but operates in an iterative manner, where it queries the retrieval system multiple times based on the generated responses until it arrives at a final answer.

The agent is an LLM model (**Llama 3.3 70B Versatile** used for our implementation), which is given tools from our retrieval system such as BM25 retrieval, Dense retrieval, Hybrid Retrieval and Hybrid Retrieval with dependency graph traversal. Additionally functions such as getting neighbouring nodes from the dependency graph and getting complete code snippet for a particular function are also provided to the agent. The agent is given complete descriptions of these tools and the signatures of these tools.

Our system is evaluated against the Agentic RAG system using **30 single-hop queries** and **20 multi-hop queries** from the ground truth dataset. The generated responses from both systems are evaluated using the same LLM-as-a-Judge evaluation metrics mentioned in the previous section. The results of the comparison are displayed in the table 7 and 8 for Context scores and table 9 and 10 for Answer scores.

The systems are evaluated on the context retrieved from the codebase and the answer generated by the system. The metrics for each are as follows:

- **Context:**
  - **Relevance:** Evaluates if the retrieved code snippets directly relate to the user query.
  - **Completeness:** Scores based on whether necessary code snippets required to answer the query are retrieved.
  - **Precision:** Measures if the retrieved code snippets are retrieved without including too much irrelevant information.
- **Answer:** Measured using the same metrics as the generation evaluation.

System	Relevance	Completeness	Precision
Agentic RAG	3.7	2.2	2.95
RepoRAG	**4.95**	**4.8**	**3.9**

Table 8: Comparison of Context of Agentic RAG system and our system using LLM-as-a-Judge evaluation metrics for multi-hop queries.

System	Faithfulness	Correctness	Completeness	Clarity
Agentic RAG	4.53	4.3	4.3	4.5
RepoRAG	**4.9**	**4.93**	**4.83**	**4.9**

Table 9: Comparison of Answers of Agentic RAG system and our system using LLM-as-a-Judge evaluation metrics for single-hop queries.

System	Faithfulness	Correctness	Completeness	Clarity
Agentic RAG	3.35	3.1	2.85	3.95
RepoRAG	**4.9**	**4.9**	**4.9**	**4.9**

Table 10: Comparison of Answers of Agentic RAG system and our system using LLM-as-a-Judge evaluation metrics for multi-hop queries.

The results are suggest that our system outperforms the Agentic RAG system in terms of both context relevance and answer quality for both single-hop and multi-hop queries.

4 Conclusion

We designed a complete retrieval pipeline for a RAG system that can effectively retrieve relevant code snippets from a code repository based on user queries. The system incorporates both sparse and dense retrieval techniques, as well as a dependency graph to capture the structural relationships between functions in the codebase. The evaluation of the retrieval system shows that the hybrid retrieval method significantly outperforms BM25-only retrieval, and the addition of the dependency graph further improves performance for multi-hop queries. In addition to this, the pipeline includes a generator that synthesizes responses based on the retrieved code context. We evaluated the generator using LLM-as-a-Judge. Along with this, we also compared our system with an Agentic RAG system, and the results suggest that our system outperforms the Agentic RAG system in terms of both context relevance and answer quality for both single-hop and multi-hop queries.